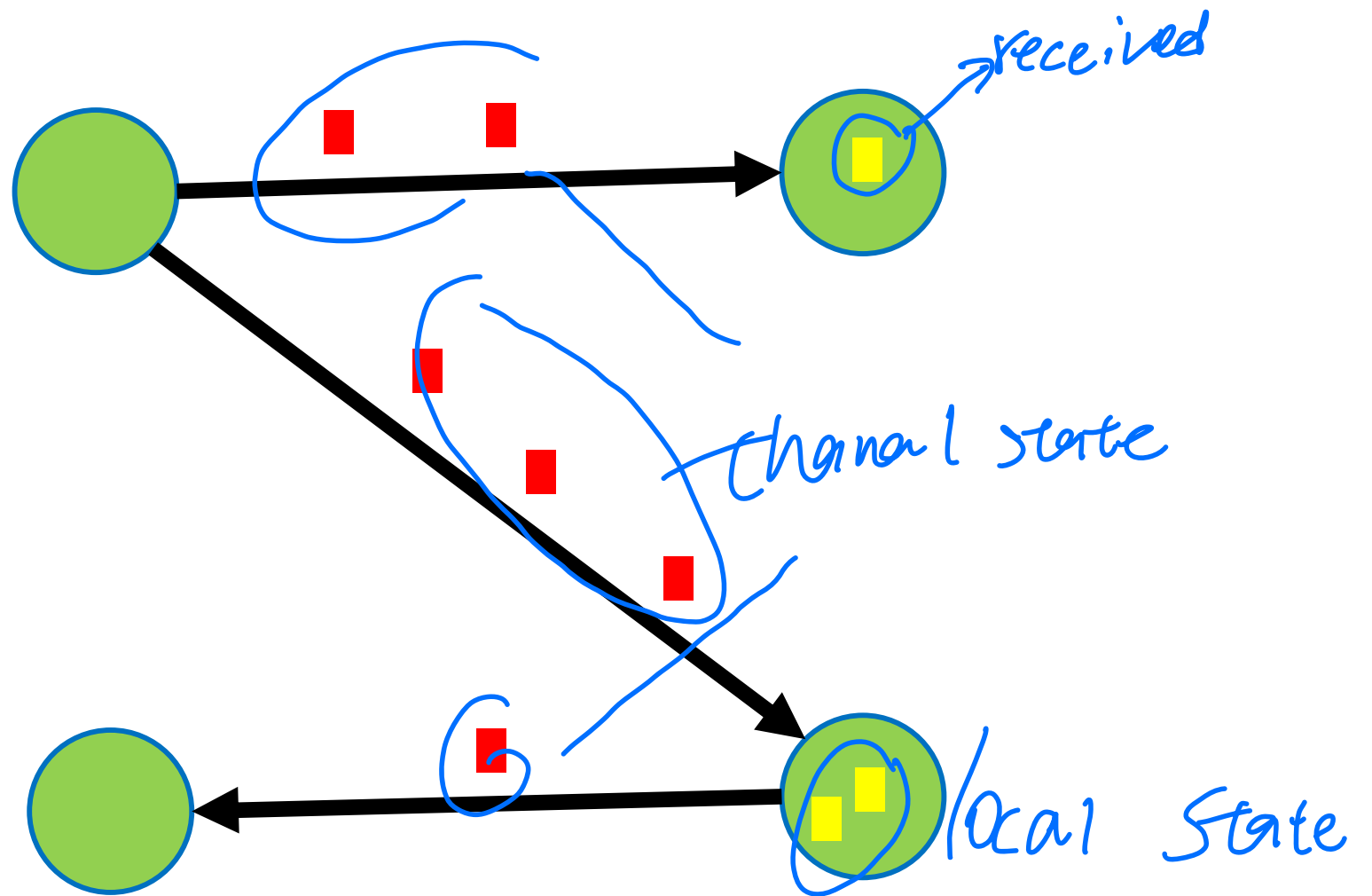


Global state and snapshot

- A distributed system consists of **processes** that communicate **asynchronously** with each other through message passing 异步通信
- Each process has a **local state** and states for each of its **communication channels**
- The **local state** of a process is the state of the process' local **memory**
- The state of a **communication channel** is the set of **messages** sent over the channel less the messages that have been received and processed by the receiver along the channel 发出—收到
- The **global state** of a distributed system is a **collection** of the **local states** of the processes and the states of the **communication channels** of all the processes



snapshot

- ❑ Recording a snapshot of a distributed system is important for many applications
 - Failure recovery
 - Debugging a distributed system
- ❑ Due to the **lack of a global clock**, obtaining a consistent global snapshot of a distributed system is not trivial
 - Example: transfer money between two accounts located at two different sites

bank system.

T1

A1 = 500

A2 = 600

T1 A1 = 500 A2 = 600

T2 A1 = 500

T1 A1 = 500 A2 = 600

T2 A1 = 500

T3 A1 = 450

Credit A2 50 → A2 = 600

T1 A1 = 500 A2 = 600

T2 A1 = 500

T3 A1 = 450

Credit A2 50 → A2 = 600

T4 Execute credit A2 50 A2 = 650

T1 A1 = 500 A2 = 600

T2 A1 = 500

T3 A1 = 450

Credit A2 50 → A2 = 600

T4 A2 = 650

T5 

T1 A1 = 500 A2 = 600

T2 A1 = 500

check point

T3 A1 = 450

Credit A2 50

A2 = 600

T4 A2 = 650

T5



T6 restored A1 = 500 restore A2 = 650

Consistent global state

□ A global state of a distributed system is consistent if the two conditions below are satisfied:

- C1: If the **sending** of a message is **recorded** in the state of the ~~sender~~, the receiving of the message must be recorded in the state (i.e., **local state or channel states**) of the receiver
- C2: Messages that have **not** been **recorded** by the **senders** should not be recorded by the receivers

保证发出和接收
状态一致

Issues in recording a global state

- ❑ **How to distinguish** between the messages to be recorded in the snapshot from those not to be recorded.
 - Any message that is sent by a process **before recording** its snapshot, **must be recorded** in the global snapshot (from C1).
 - Any message that is sent by a process **after recording** its snapshot, **must not be recorded** in the global snapshot (from C2).
- ❑ **How to determine** the instant when a process takes its snapshot.
 - A process p_j must **record its snapshot before processing** a message m_{ij} that was sent by process p_i **after** recording its snapshot.

T1 A1 = 500 A2 = 600

T2 A1 = 500

T3 A1 = 450

Credit A2 50
A2 = 600

T4 A2 = 650

T5 

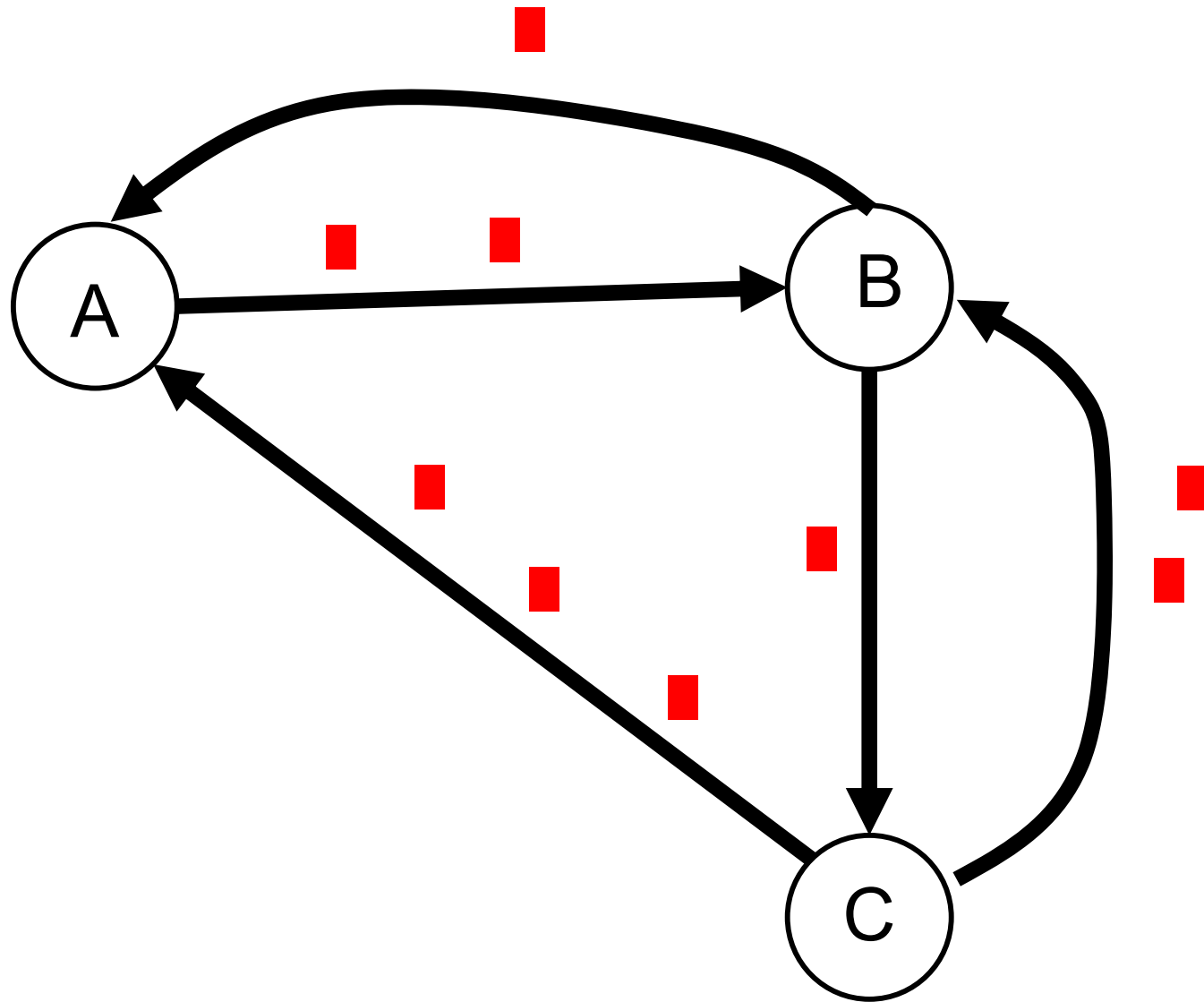
T6 restored A1 = 500 restore A2 = 650

Snapshot algorithm for FIFO channels

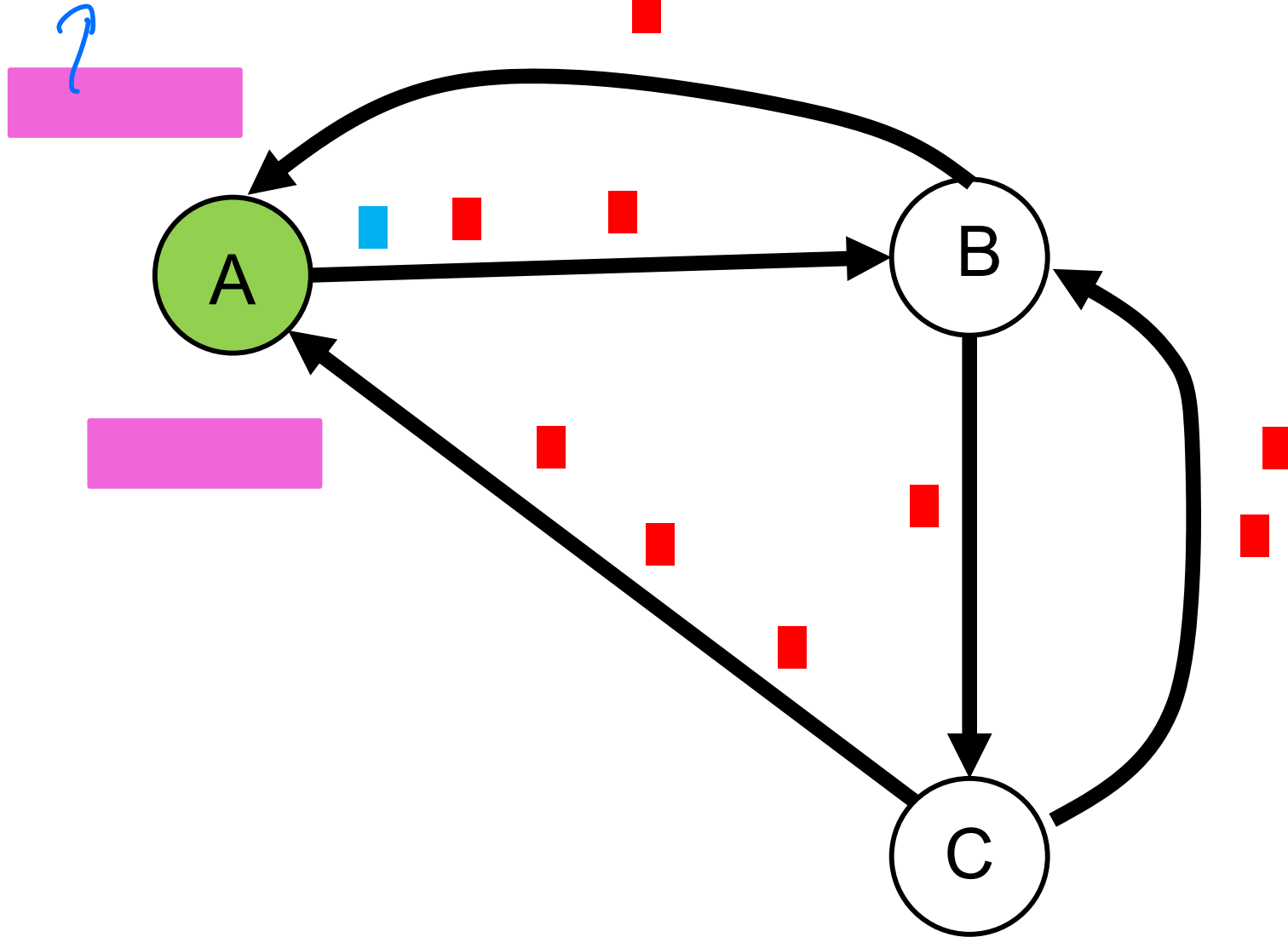
所有的 process 收到 marker message, record, 并向所有 发出 marker message

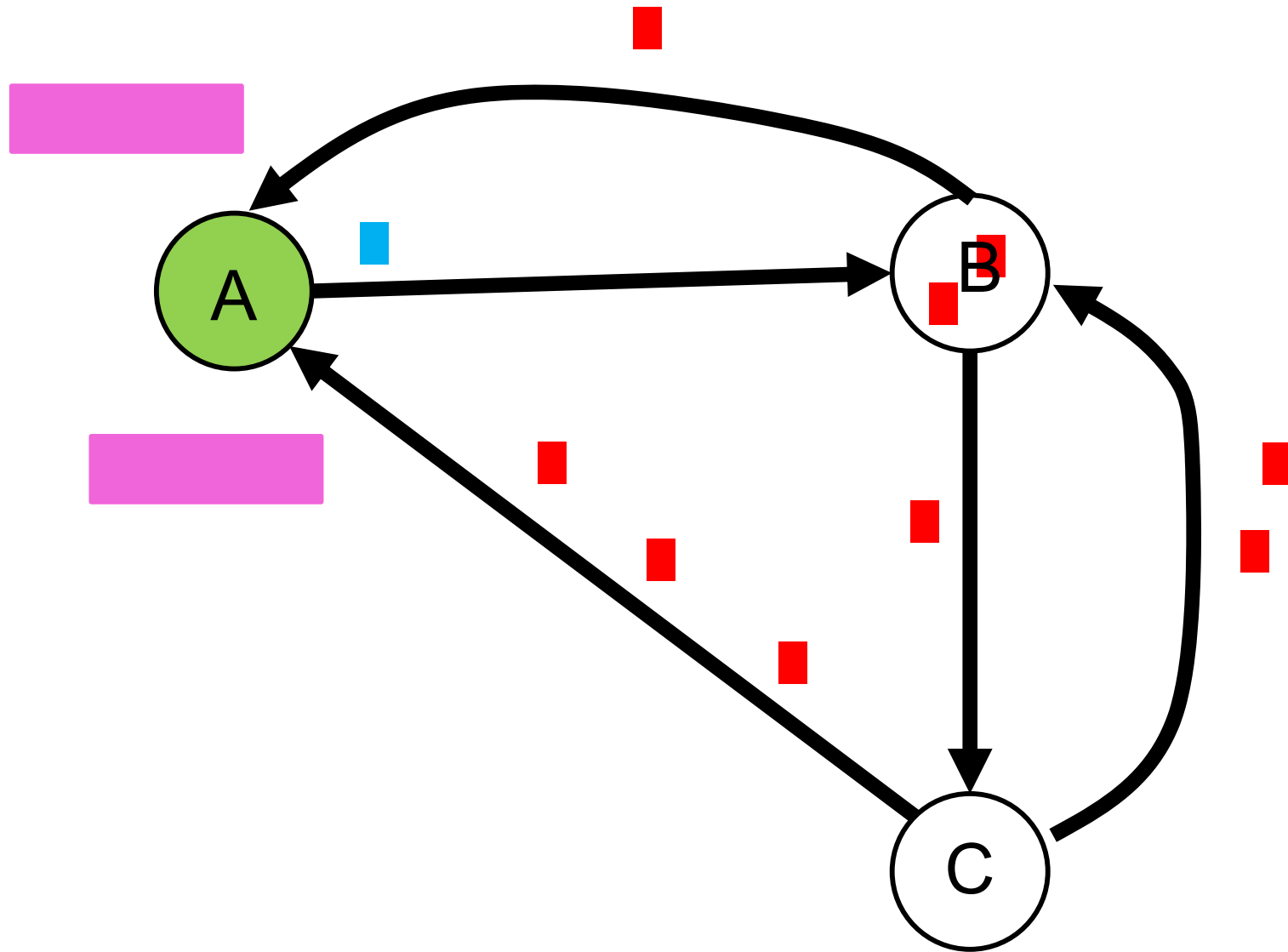


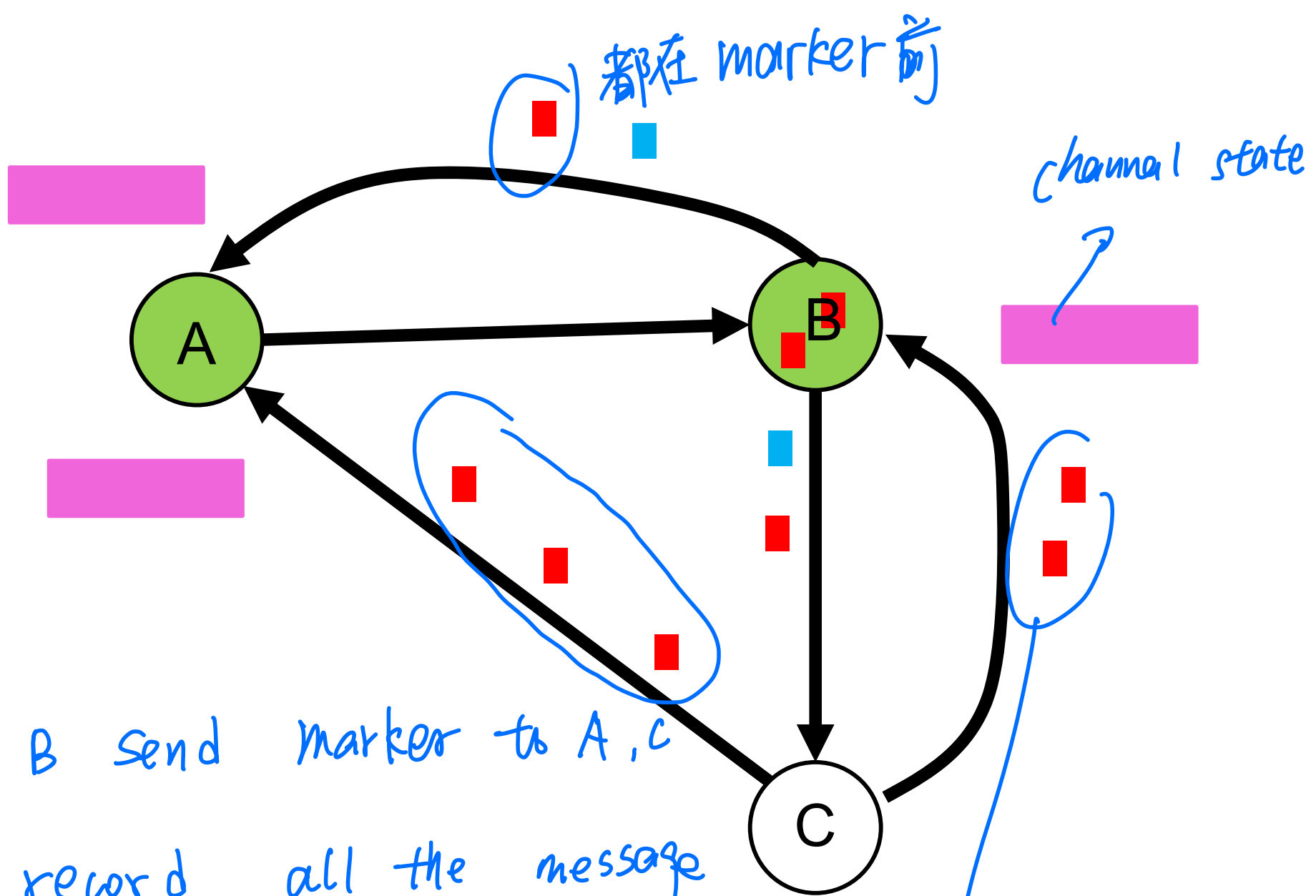
- The Chandy-Lamport algorithm uses a control message, called a **marker** to separate messages in the channels.
 Check 是否在发出消息前 record
- **After** a site has recorded its **snapshot**, it sends a marker, along all of its outgoing channels before sending out any more messages.
- A marker **separates** the messages in the channel into those to be included in the snapshot from those not to be recorded in the snapshot.
- A process should **record** its snapshot when it receives the first marker on any of its incoming channels.
 收到 marker 记录快照



queue for record.





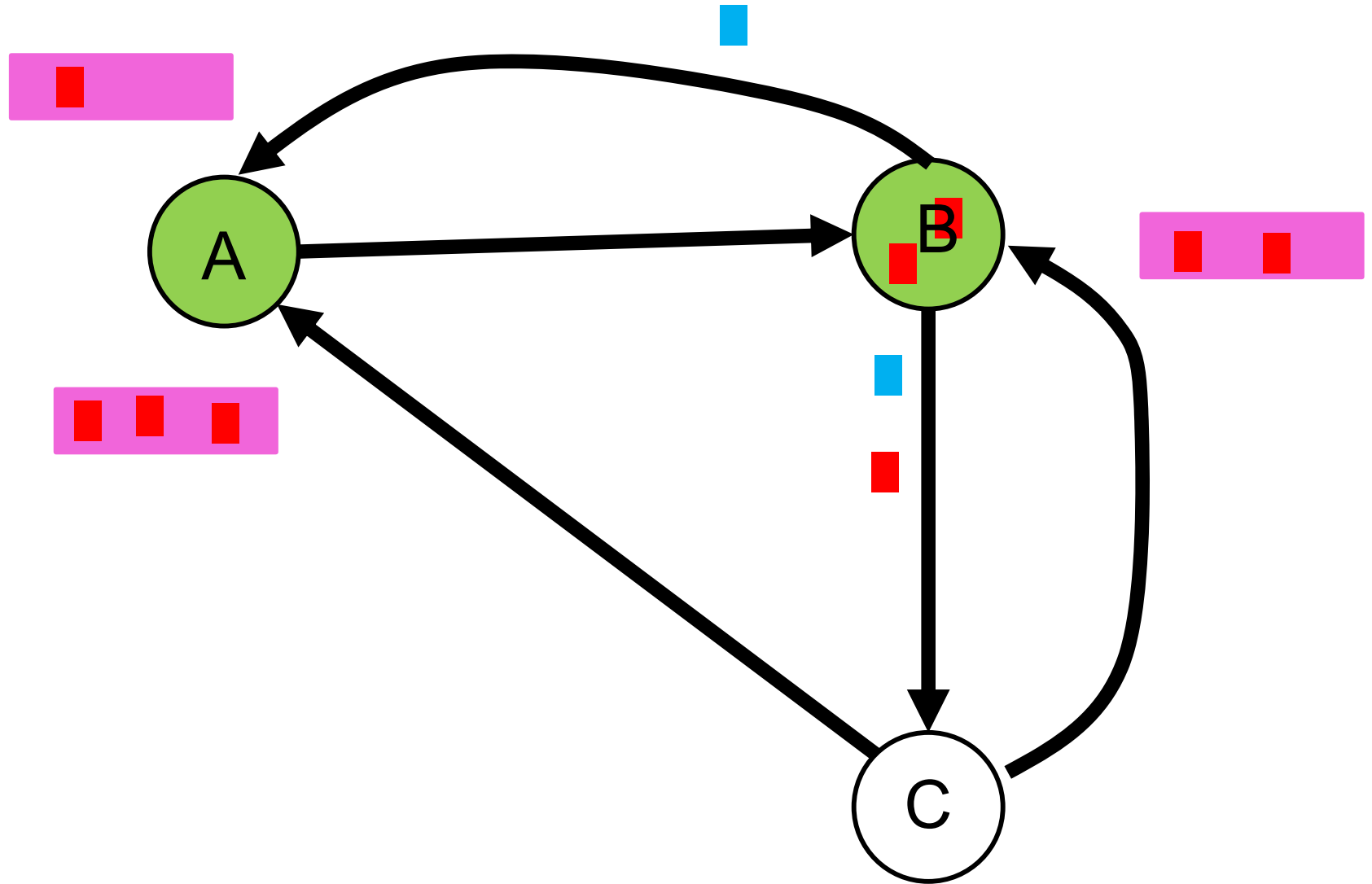


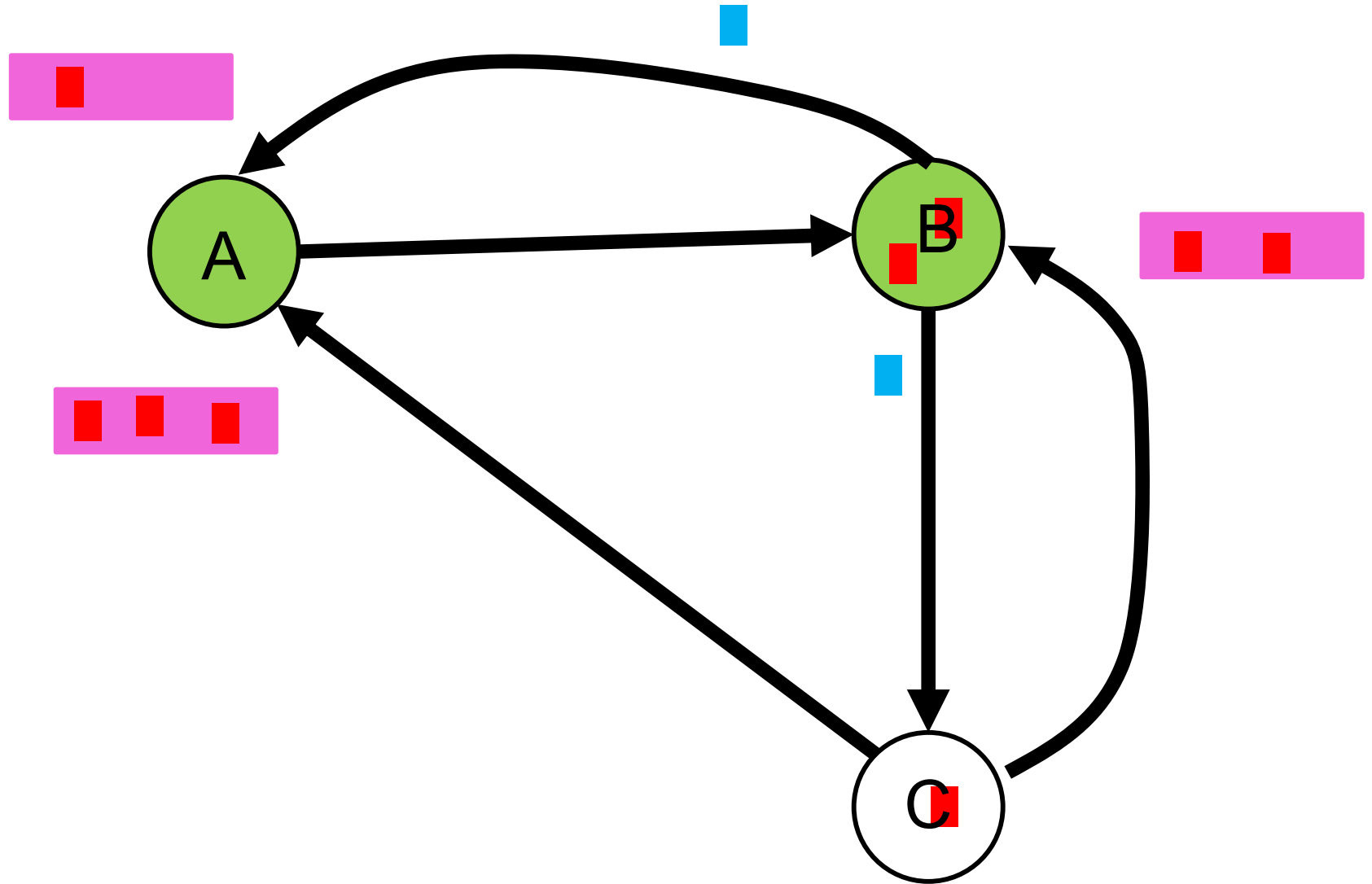
① B send marker to A, C

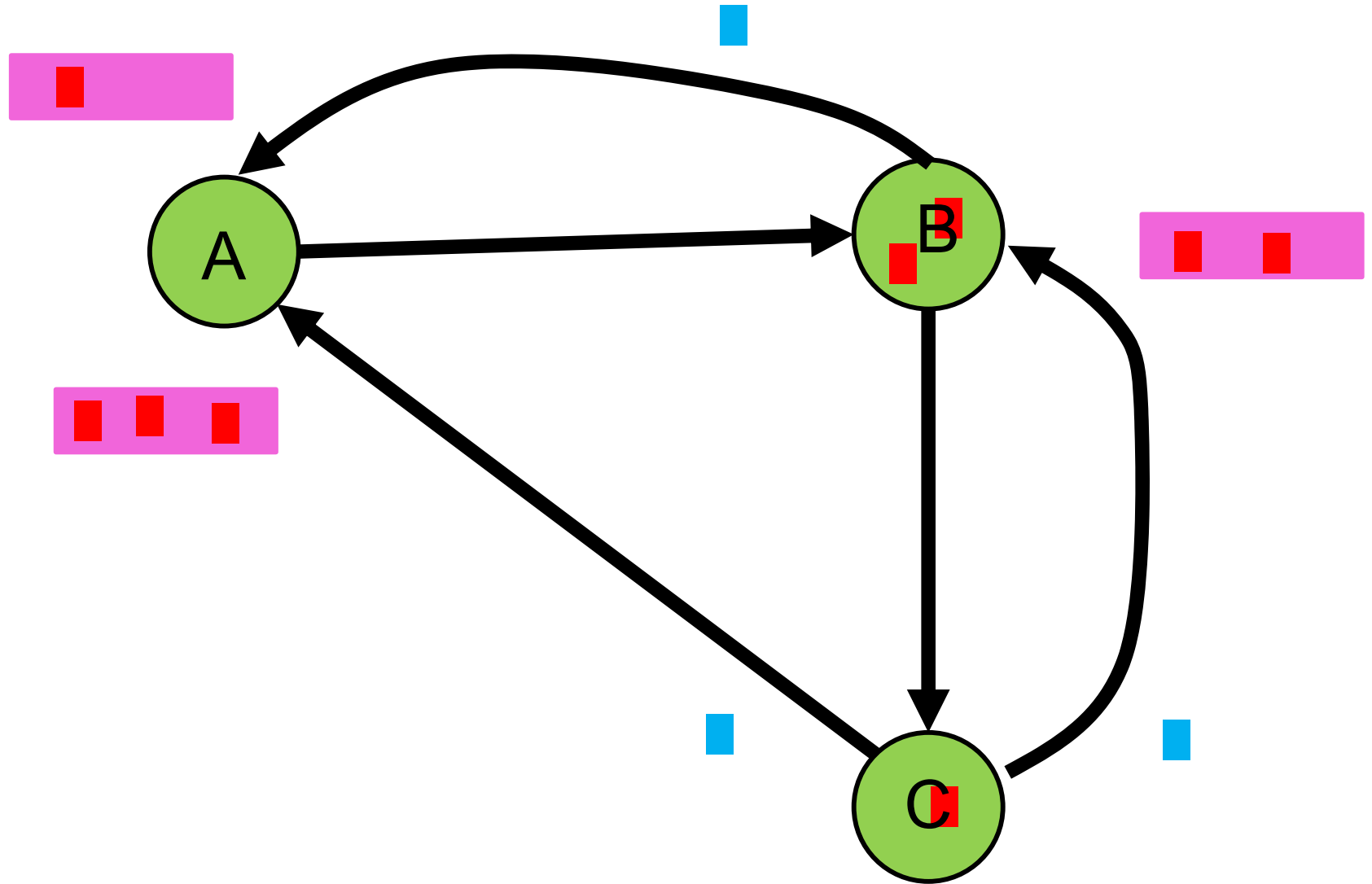
② record all the message

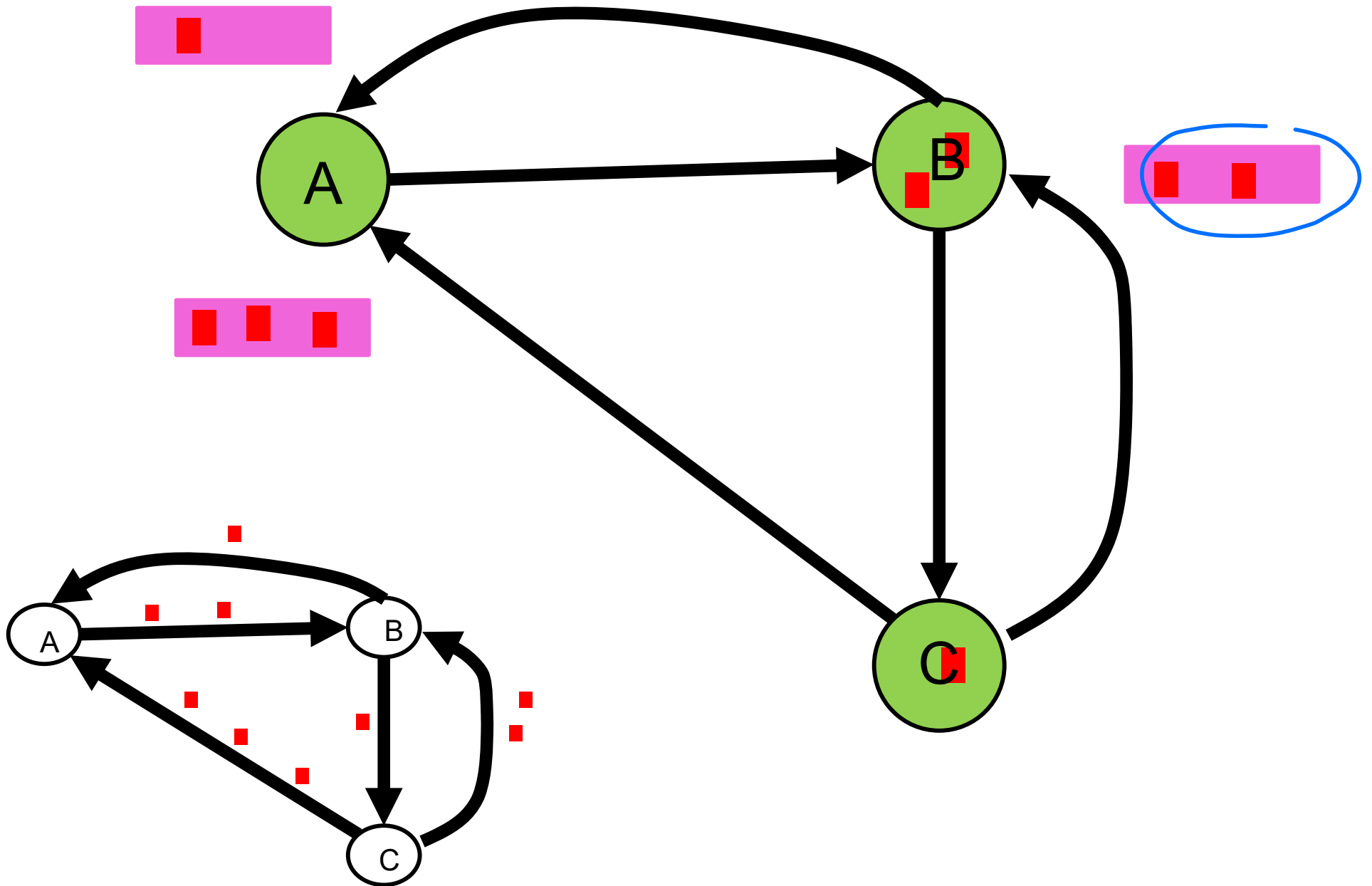
all the income channel

因为在 C 发出 marker 之后，需要记录









- ❑ The algorithm can be initiated by any process by executing the “Marker Sending Rule” by which it records its local state and sends a marker on each outgoing channel.
- ❑ A process executes the “Marker Receiving Rule” on receiving a marker. If the process has not yet recorded its local state, it records the state of the channel on which the marker is received as empty and executes the “Marker Sending Rule” to record its local state.
- ❑ The algorithm terminates after each process has received a marker on all of its incoming channels.
- ❑ All the local snapshots get disseminated to all other processes and all the processes can determine the global state.

Marker Sending Rule for process i

- Process i records its state.
- For each outgoing channel C on which a marker has not been sent, i sends a marker along C

Marker Receiving Rule for process j

On receiving a marker along channel C :

- if j has not recorded its state then
 - Record the state of C as the empty set
 - Follow the “Marker Sending Rule”
- else
 - Record the state of C as the set of messages received along C after j 's state was recorded and before j received the marker along C

Correctness

- Due to FIFO property of channels, it follows that no message sent **after the marker** on that channel is recorded in the channel state. Thus, condition **C2** is satisfied.
- When a process p_j receives message m_{ij} that precedes the marker on channel C_{ij} , it acts as follows:
 - If process p_j **has not taken its snapshot** yet, then it includes m_{ij} in its recorded snapshot.
 - Otherwise, it **records** m_{ij} in the state of the channel C_{ij} .
 - Thus, condition **C1** is satisfied.

reviews

- ❑ Why is obtaining a consistent global snapshot difficult in a distributed system? Use an example to support your argument.
- ❑ What are the conditions that need to be satisfied to make a global state consistent?
- ❑ For some systems, the Chandy-Lamport algorithm can be modified so that all the channel states are recorded as empty. Which property do these systems need to possess and how Chandy-Lamport algorithm should be modified so that the algorithm does not need to record the channel state?
- ❑ Modify the Chandy-Lamport algorithm so that it handles the case in which several sites initiates the snapshot algorithm simultaneously. You should minimize the amount of messages being exchanged as much as possible.

readings

- ❑ K.M. Chandy and L. Lamport, Distributed snapshots: determining global states of distributed systems, *ACM transactions on Computer Systems*, 3(1), 1985, 63-75
- ❑ M. Spezialetti and P. Kearns, Efficient distributed snapshots, *Proceedings of the 6th Intl. Conf. on distributed computing systems*, 1986, 382-388